PYTHON FOR DATA SCIENCE

NumPy Cheat Sheet

Roman Urdu Comments ke Saath

by
Syed Sufiyan Jawaid

> NumPy Kya Hai?

NumPy Python ka ek powerful scientific computing package hai. Isme **ndarray** (n-dimensional array) hota hai jo:

- Fast mathematical operations karta hai
- Saare elements ek hi type ke hote hain (homogeneous)
- Python lists se zyada fast aur memory efficient hota hai
- Array ka size creation ke waqt fix hota hai
- Agar size badalna ho to naya array create hota hai

> Import NumPy

```
>>>
# NumPy library import karo aur s
hortcut naam "np" diya
>>> import numpy as np
```

> Creating Arrays

Array Creation Methods — np.array, arange, ones, zeros, random

```
>>>
# 1D array — Ek seedhi line mein
values store hoti hain
>>> a = np.array([1,2,3])
>>>
# 2D array — Rows aur columns wal
i table (2 rows, 3 columns)
>>>
b = np.array([[1,2,3],[4,5,6]])
>>>
```

> Array Properties

Understanding ndim, shape, size, dtype

```
>>>
# 1D array — 0 se 9 tak values
>>>
a1 = np.arange(10, dtype=np.int3
2)
>>>
# 3 rows aur 4 columns — Float ty
pe values
>>>
a2 = np.arange(12, dtype=float).r
eshape(3,4)
```

Syed Sufiyan Jawa

```
# 3D array — Pages + rows + columns (multiple tables ek saath)
>>>
c = np.array([[[1,2],[3,4]],[[5,6],[7,8]]])
>>>
# dtype se data type specify — int ko float mein convert
>>>
np.array([1,2,3], dtype=float)
>>>
# 1 se 10 tak, Step=2 — Output: 1,3,5,7,9
>>> np.arange(1,11,2)
>>>
# 0 se 15 tak — reshape se dimensions change (elements same)
>>>
np.arange(16).reshape(2,2,2,2)
>>>
# 3 rows aur 4 columns — Har jagah 1.0 fill hoga
>>> np.ones((3,4))
>>>
# 3x4 array — Har jagah 0.0 fill hoga
>>> np.zeros((3,4))
>>>
# Random decimal numbers — Range: 0 se 1 ke beech
>>> np.random.random((3,4))
>>>
# -10 se 10 tak — Exactly 10 equal distance wali values
>>>
np.linspace(-10,10,10, dtype=int)
>>>
# 3D array — 2 pages, 2 rows, 2 columns
>>>
a3 = np.arange(8).reshape(2,2,2)
>>>
# Array kitne dimensions ka hai — Output: 3
>>> a3.ndim
>>>
# Array ki exact shape batata hai — Output: (2,2,2)
>>> a3.shape
>>>
# Total elements count karega — 3 x 4 = 12
>>> a2.size
>>>
# Ek element kitni memory leta hai — int64 = 8 bytes
>>> a3.itemsize
>>>
# Array ka data type — Example: int32, float64
>>> a1.dtype
>>>
# Array ka type convert karo — Float -> int32
>>> a3.astype(np.int32)
```

> Array Operations

Scalar and Vector Operations

```
>>> # 3x4 array — Values: 0 se 11
>>>
a1 = np.arange(12).reshape(3,4)
>>> # 12 se 23 tak values
>>>
```

ed Sufiyan Jawa

```
# Identity matrix — Diagonal pe 1, baaki sab jagah 0
>>> np.identity(3)
```

```
a2 = np.arange(12,24).reshape(3,4)
>>>
# Har element ka square — Scalar operation
>>> a1 ** 2
>>>
# Check kahan value 15 hai — Output: True/False
>>> a2 == 15
>>>
# Same position wale elements add — shape same honi chahiye
>>> a1 + a2
>>>
# Same position wale elements multiply honge
>>> a1 * a2
>>>
# a1 ka element, a2 ke corresponding element ki power
>>> a1 ** a2
```

> Array Functions

Mathematical and Statistical Functions

```
>>>
# Columns ke andar multiply — upar se neeche operation
>>> np.prod(a1, axis=0)
>>>
# Rows ka sum — left se right operation
>>> np.sum(a1, axis=1)
>>> # Sabse bari value
>>> np.max(a1)
>>> # Sabse chhoti value
```

> Indexing and Slicing

Accessing Data from Arrays

```
>>>
# 3 rows aur 4 columns wali table
>>>
a2 = np.arange(12).reshape(3,4)
```

Single Element Access

```
>>>
# Row 1 = doosri row, Column 0 = pehla column
>>> a2[1,0]
>>>
```

ed Sufiyan Jawa

```
>>> np.min(a1)
>>> # Average value
>>> np.mean(a1)
>>> # Middle value after sorting
>>> np.median(a1)
>>>
# Standard deviation — values average se kitni door hain
>>> np.std(a1)
>>>
# Variance row wise calculate karega
>>> np.var(a1, axis=1)
>>> # Sine function apply karega
>>> np.sin(a1)
>>> # e^x calculate karega
>>> np.exp(a1)
>>>
# Dot product / Matrix multiplication (columns = rows rule)
>>> np.dot(a2, a3)
>>>
# Upar ki taraf round — 4.1 -> 5
>>> np.ceil(x)
>>>
# Neeche ki taraf round — 4.9 -> 4
>>> np.floor(x)
>>> # Nearest integer pe round
>>> np.round(x)
```

```
# 3D — Page 1, Row 0, Column 1
>>> a3[1,0,1]
```

Slicing Format

```
[start : stop : step]
start = kahan se shuru karna hai
stop = kahan tak jana hai
step = kitna jump karna hai
```

```
>>>
# Index 2 se 4 tak, Step 2 = ek value skip
>>> a1[2:5:2]
```

2D Slicing Examples

```
>>>
# 0:2 = row 0 aur 1, 1::2 = column 1 se, har doosra
>>> a2[0:2, 1::2]
>>>
# Doosri row, har teesra column
>>> a2[1, ::3]
>>> # Pehli row ki saari values
>>> a2[0, :]
>>>
# Teesra column, saari rows ke sath
>>> a2[:, 2]
>>>
# Row 1 se end tak, Column 1 aur 2
>>> a2[1:, 1:3]
```

3D Slicing

```
>>>
# 3 pages, har page mein 3x3 table
>>>
```

ed Sufiyan Jawa

```
a3 = np.arange(27).reshape(3,3,3)
>>>
# Page 0 aur 2, Row 0, Column 0 aur 2
>>> a3[::2, 0, ::2]
>>>
# Page 2, Row 1 se end, Column 1 se end
>>> a3[2, 1:, 1:]
>>>
# Page 0, Row 1, Saare columns
>>> a3[0, 1, :]
```

> Iterating Arrays

Looping Through Arrays

```
>>>
# 1D array — har element alag print hoga
>>> for i in a1:
...     print(i)
>>>
# 2D array — har row print hoti hai
>>> for i in a2:
...     print(i)
>>>
# nditer — 3D array ka bhi har element access
>>>
for i in np.nditer(a3):
...     print(i)
```

> Reshaping Arrays

Changing Array Shapes

```
>>>
# Rows aur columns swap — 3x4 -> 4x3
>>> np.transpose(a2)
>>>
# transpose ka short cut
>>> a2.T
>>>
# Kisi bhi shape ko 1D line mein convert karo
>>> a3.ravel()
```

> Stacking Arrays

Combining Multiple Arrays

```
>>>
a4 = np.arange(12).reshape(3,4)
>>>
a5 = np.arange(12,24).reshape(3,4)
>>>
# Horizontal — side by side, 3x4+3x4=3x8
>>>
np.hstack((a4, a5))
>>>
# Vertical — upar neeche, 3x4+3x4=6x4
>>>
np.vstack((a4, a5))
```

> Splitting Arrays

ed Sufiyan Sawa

Dividing Arrays into Parts

```
>>>
# Columns ke hisaab se — Horizontal splitting
>>> np.hsplit(a4, 2)
>>>
# Rows ke hisaab se — Vertical splitting
>>> np.vsplit(a5, 2)
```

> Quick Reference

Important Concepts at a Glance

`axis=0`	Columns ke saath operation (upar neeche)
`axis=1`	Rows ke saath operation (left right)
`[row, col]`	Hamesha row pehle, column baad mein
`:`	Sab kuch select karo
`reshape()`	Total elements same rehne chahiye
`ravel()`	Kisi bhi shape ko 1D flat array bana deta hai
`.T`	Transpose shortcut
`ndim`	Dimensions count
`shape`	Rows/columns structure
`size`	Total elements
`dtype`	Data type

> Important Tips

- NumPy arrays Python lists se fast hote hain
- Vectorized operations loops se zyada efficient hote hain
- Shape mismatch hone par operations error denge
- reshape() karte waqt total elements same rehne chahiye
- axis=0 ko columns aur axis=1 ko rows samjho

Short Revision Line

`NumPy = Fast arrays + Fast math + Easy data manipulation`

Syed Sufiyan Jawa